APPENDIX A

Setting Up and Verifying the Examples

All the code in this book lives in the `code/` directory of the book's repository: a Cargo workspace with no dependencies. This appendix explains how to install the right compiler, run the examples, compile the programs that are supposed to fail, and use the verification script, including how to use it to see what changes when you try a newer compiler.

In this appendix you will:

- install the pinned toolchain with rustup
- run an example and compile a program that must be rejected
- verify the whole book with `tools/run_examples.py`
- compare the book's diagnostics with those of another compiler version

A.1 Installing the toolchain

Install Rust with rustup if you have not already. The workspace pins its compiler in `code/rust-toolchain.toml`:

```
# The book shows real compiler diagnostics, so the exact compiler is pinned.
# rustup installs it automatically the first time you run cargo here.
# rust-src is the standard library's source: rust-analyzer needs it, and with it
# installed rustc quotes standard-library code in some notes, which the
# diagnostic snapshots contain.
[toolchain]
channel = "1.99.0"
components = ["rustfmt", "rust-src"]
```

You do not need to install Rust 1.99 by hand: the first time you run `cargo` or `rustc` inside `code/`, rustup reads this file and installs the toolchain with the two listed components. `rust-src` is the source of the standard library. Editors need it to navigate into the standard library, and with it installed rustc quotes standard-library code in a few notes, which is what the book's diagnostics show. To check which compiler is active:

```
cd code
rustup show active-toolchain   # 1.99.0-... (overridden by '.../code/rust-toolchain.toml')
rustc --version                # rustc 1.99.0 (b940084d7 2026-09-28)
```

If you use rust-analyzer, note that the workspace is one directory below the repository root. The repository contains a `.vscode/settings.json` that points rust-analyzer at `code/Cargo.toml`; for other editors, open `code/` as the project, or configure the linked project the same way.

A.2 Running the examples

Runnable examples are single files in `code/examples/`, named after the chapter that uses them: `ch09_callee_picks.rs` belongs to Chapter 9. Each one starts with a line such as `// harness: run` (for the verification script, see below) and a doc comment that says what it shows and how to run it:

```
cd code
cargo run --example ch09_callee_picks
```

Every example builds without warnings and prints the output that the book shows. Change them freely: they are small, and most of the exercises start from one.

A.3 Compiling the programs that must be rejected

The programs in `code/compile_fail/` are not part of the Cargo package (they would stop it from building). Each one is a small library, with no `main` function, that the compiler must reject. Its first line lists the error codes or messages it must produce, for example:

```
// harness: compile-fail E0597
```

and its diagnostics are stored next to it in a file with the extension `.stderr`. To see the errors yourself, compile one with `rustc` from the `code/` directory, the same way the verification script does:

```
cd code
rustc --edition 2024 --crate-type lib --emit=metadata -A warnings \
      -o /tmp/out.rmeta compile_fail/ch09_caller_picks.rs
```

`--crate-type lib` says that there is no `main` to look for, `--emit=metadata` stops after type and borrow checking (no code is generated), and `-A warnings` hides warnings that are not part of the lesson, such as unused variables. A few programs declare an older edition in their first line (`edition=2021`); compile those with `--edition 2021` to see what the book shows.

A.4 Verifying everything

`tools/run_examples.py` is the script that keeps the book honest. From the root of the repository:

```
python3 tools/run_examples.py                 # every example and compile-fail program
python3 tools/run_examples.py --only ch09     # one chapter
python3 tools/run_examples.py --rust          # only the runnable examples
python3 tools/run_examples.py --compile-fail  # only the programs that must fail
```

For each runnable example, it builds the example (failing on any warning), runs it, and compares its standard output with the snapshot in `code/output/`, which is what the book includes. For each compile-fail program, it compiles the file, checks that compilation fails with the declared codes or messages, and compares the complete diagnostics with the `.stderr` snapshot. A line of output per program and a final count tell you whether everything matches:

```
ok   rust          ch09_callee_picks                    0.4s
ok   compile-fail  ch09_caller_picks                    0.0s
```

```
...
2 passed, 0 failed
```

When something differs, the script prints a diff between the snapshot and what the compiler produced. Two options change what it does with differences:

- `--bless` rewrites the snapshots from the current results. Authors use it after changing an example; you can use it after an experiment, to see the new diagnostics in the files.
- `--codes-only` checks only that each compile-fail program fails with its declared codes and messages, not that the full text is identical.

A.5 Trying a newer (or older) compiler

Diagnostics change from release to release, usually for the better. To see how your compiler differs from the one the book was verified with, override the pinned toolchain for one run with the `RUSTUP_TOOLCHAIN` environment variable, which takes precedence over `rust-toolchain.toml`:

```
RUSTUP_TOOLCHAIN=stable python3 tools/run_examples.py --codes-only
RUSTUP_TOOLCHAIN=stable python3 tools/run_examples.py --compile-fail
```

The first command answers the important question: does every program that the book says is rejected still fail for the same reason, and does every example still compile and print the same output? The second shows the exact differences in wording, which are usually harmless (a reworded label, an extra help line). If a program that should fail now compiles, the language has changed. The most likely candidate is the example of Chapter 5 that today's borrow checker rejects and Polonius accepts (see the next section).

A.6 Nightly-only experiments

A few places in the book mention features that are only available on the nightly compiler, most importantly Polonius, the borrow checker that is expected to replace today's NLL checker, which accepts some programs that stable Rust rejects (Chapter 5). To try them, install a nightly toolchain and compile the program with it explicitly:

```
rustup toolchain install nightly
cd code
rustc +nightly --edition 2024 --crate-type lib --emit=metadata \
      -o /tmp/out.rmeta compile_fail/ch05_nll_case3.rs
```

On stable Rust 1.99 this program is rejected with E0502 (that is why it is in `compile_fail/`); with the nightly of 2026-10-08 it compiles. Adding `-Zpolonius=off` to the nightly command switches Polonius off again, and the error comes back.

The book's statements about nightly behavior were checked with `rustc 1.101.0-nightly (2026-10-08)`. Nightly changes every day: if the result differs, check the status of the feature in its tracking issue (Appendix E lists them).

A.7 Building the book

You do not need to build the book to use it, but if you want to rebuild the PDFs after changing an example, the repository's README lists the requirements (Python 3.11, pandoc 3, Typst and Graphviz) and the commands. The short version:

```
python3 tools/build_book.py check   # includes, anchors and image links
python3 tools/build_book.py all     # diagrams, per-chapter Markdown and PDFs, the book PDF
```

A.8 Summary

- The compiler is pinned in `code/rust-toolchain.toml` (Rust 1.99.0, edition 2024); rustup installs it on first use.
- Run an example with `cargo run --example chNN_name` from `code/`.
- Compile a rejected program with `rustc --edition 2024 --crate-type lib --emit=metadata -A warnings <file>`.
- `python3 tools/run_examples.py` verifies every program, output and diagnostic; `--only`, `--bless` and `--codes-only` select, update and relax the checks.
- `RUSTUP_TOOLCHAIN=<toolchain>` runs the verification with another compiler.